CHEAT SHEET · JS INTERVIEW #13 @specsycoder

NaN === NaN

The only value not equal to itself

nan.js

```
1  console.log(NaN === NaN); // false
2  console.log(typeof NaN); // "number"
```

output `false` `"number"`

Why it happens

01 Not a Number
NaN is the result of math that failed.

02 Still a number type
typeof NaN is "number". It lives in the number system.

03 Never equal
Two failed results are not the same, so NaN equals nothing.

04 Not even itself
NaN === NaN and NaN == NaN are both false.

Where NaN comes from

where-nan-comes-from.js

```
1  0 / 0; // NaN
2  Number("hello"); // NaN
3  parseInt("abc"); // NaN
4  Math.sqrt(-1); // NaN
```

Save this · share with a friend Follow @specsycoder for daily JS interview Qs

@specsycoder

How to check NaN

Number.isNaN vs isNaN vs Object.is

check.js

```
1  Number.isNaN(NaN); // true
2  Number.isNaN("hello"); // false
3  isNaN("hello"); // true (!)
4  Object.is(NaN, NaN); // true
```

expression	result	why
[NaN].includes(NaN)	true	includes uses SameValueZero
[NaN].indexOf(NaN)	-1	indexOf uses ===
NaN + 5	NaN	NaN spreads through math
x !== x	true only for NaN	old-school check

Traps to mention

- **Global isNaN converts first.** isNaN("hello") is true. Use Number.isNaN.
- **One NaN poisons a sum.** A single bad input turns the whole total into NaN.

Your 20-second answer

"NaN is the result of failed math, and its type is still number. By the spec, NaN is not equal to anything, not even itself, so NaN === NaN is false. To check for it, I use Number.isNaN, because the global isNaN converts the value first."